

Supón que tienes un robot que aspira cuartos sucios, el cual puede moverse a la *izquierda* o *derecha*, *aspirar* o *no hacer nada*. Tienes dos cuartos A y B . Tus robot tiene que considerar que cuartos están sucios o limpios y en que cuarto se localiza.

- Tu estado sería:

$$S = (A, B, R)$$

Donde:

$A \in \{L, S\} =$ (El cuarto A esta Limpio o Sucio)

$B \in \{L, S\} =$ (El cuarto B esta Limpio o Sucio)

$R \in \{A, B\} =$ (El Robot se encuentra en A o en B)

- Tus acciones serían:

$$A = \{Izq, Der, Asp, Nada\}$$

- Tu percepción sería:

$$\rho = (R, S[R]) = (\text{Cuarto a ir, Cuarto en donde se encuentra el robot})$$

Ahora, queremos que el robot limpie de la forma mas eficientes y que no se quede ciclado al tomar decisiones.

AQUI PON ESTADOSROBOT.EXCALIDRAW CUANDO LO TERMINES con la foto que tomaste en el cel

Agentes Reflejo

Sus acciones se definen así:

- $a_t = f(P_t)$
"Toma acciones en un tiempo t (a) dependiendo de una percepción en un tiempo t (P_t)"
- $a_t = f(P_t, X_t)$
"Toma acciones en un tiempo t (a) dependiendo de una percepción en un tiempo t (P_t) y del estado interno del robot en un tiempo t (X_t)". Básicamente, el modelo mantiene memoria de sus acciones pasadas. En este modelo, es necesario inicializar el estado S con el estado del robot inicial X_t .

Ejemplo hipotético en código

```
xt = (S, S, A)
```

```
def f(Pt, Xt):
```

```
# Actualizar Xt
Xt[2] = Pt[0]
if Pt[0] == 'A':
    Xt[0] = Pt[0]
else:
    Xt[1] = Pt[1]

if Xt[0] == Xt[1] == 'L':
    a = 'Nada'
elif Pt[1] == 'S':
    a = 'Aspira'
elif Pt[0] == 'A':
    a = 'Derecha'
else:
    a = 'Izquierda'

return a
```